

Problem 1 (Bonus) :

- Run a container nginx with name my-nginx and attach a and attach a volume 2 volumes to the container
 - Volume1 for containing static html file
 - Volume2 for containing nginx configuration
- Edit the html content
- Remove the container
- Run a new 2 containers with the following:
 - Attach the 2 volumes that was attached to the previous container in two different ways (volume mount – bind mount)
 - Map port 80 to port 8080 on you host machine
 - Access the html files from your browser

```

root@7d3431b12222: /
ahmed@HP:home/ahmed$ docker volume create vol-1
vol-1
ahmed@HP:home/ahmed$ docker volume create vol-2
vol-2
ahmed@HP:home/ahmed$ docker run -v vol-1:/usr/share/nginx/html -v vol2:/etc/nginx/conf.d -p 9090:8090 -d nginx
1faebf34300121e121a7886bf3bfd9b7ce1fd1d38bca30d5d606a369f1533545
ahmed@HP:home/ahmed$ docker exec -it 1faebf34300121 bash
root@1faebf343001:/# echo 'Ahmed Ehab Ahmed' > /usr/share/nginx/html/index.html
root@1faebf343001:/# vi /etc/nginx/conf.d/default.conf
bash: vi: command not found
root@1faebf343001:/#

```

apt-get update

apt-get install vim -y

vim /etc/nginx/conf.d/default.conf

```

root@7d3431b12222: /
server {
    listen      8090;
    listen [::]:8090;
    server_name localhost;

    #access_log /var/log/nginx/host.access.log main;
}

```

exit

docker rm -f 1faebf343001

mkdir -p ~/my-nginx-config

sudo cp -r /var/lib/docker/volumes/vol-2/_data/. ~/my-nginx-config/

```

root@7d3431b12222: /
root@7d3431b12222: /
ahmed@HP:home/ahmed$ docker run -v vol-1:/usr/share/nginx/html -v ~/my-nginx-config:/etc/nginx/conf.d -p 9090:8090 -d nginx
d2a9273939d49acb5bc989078f6e51240f49ec812d18736aabc111e1e658c618
ahmed@HP:home/ahmed$ curl localhost:9090
Ahmed Ehab Ahmed
ahmed@HP:home/ahmed$

```

Problem 2:

- Create a dockerfile for nginx image with different html content and different nginx conf that listen to port 8080 instead of port 80 on the container
- Create container from the new image

```
dockerfile x
dockerfile > ...
1 FROM nginx
2
3 RUN echo "Hello, OS46!" >/usr/share/nginx/html/index.html
4
5 COPY default.conf /etc/nginx/conf.d/default.conf
6
7 EXPOSE 8080
8
```

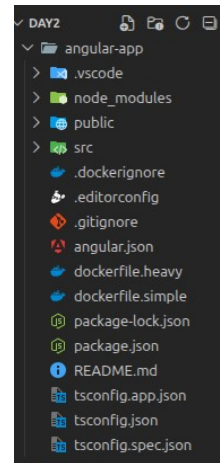
```
ahmed@HP: /mnt/0844D3E544D3D392/CS/ITI/Docker/Day2
$ docker build -t my-nginx .
[+] Building 2.4s (8/8) FINISHED                                docker:default
=> [internal] load build definition from dockerfile             0.1s
=> => transferring dockerfile: 172B                             0.0s
=> [internal] load metadata for docker.io/library/nginx:latest 0.0s
=> [internal] load .dockerignore                               0.0s
=> => transferring context: 2B                                    0.0s
=> [1/3] FROM docker.io/library/nginx:latest@sha256:0236ee02dcfce00b9bd8 0.2s
=> => resolve docker.io/library/nginx:latest@sha256:0236ee02dcfce00b9bd8 0.0s
=> [internal] load build context                                0.1s
=> => transferring context: 1.14kB                                0.0s
=> [2/3] RUN echo "Hello, OS46!" >/usr/share/nginx/html/index.html 0.3s
=> [3/3] COPY default.conf /etc/nginx/conf.d/default.conf      0.1s
=> exporting to image                                           1.5s
=> => exporting layers                                           1.2s
=> => exporting manifest sha256:c0a537ac2e2bf41bb9eadf2a92233ad098f3d175 0.1s
=> => exporting config sha256:8752dd36e55910e1ae4e809c2b9eadbb1d324905c3 0.1s
=> => exporting attestation manifest sha256:672a94567ae0fad8efe5ea062d98 0.1s
=> => exporting manifest list sha256:aef8c3a64069ca928128a5bf96453f6eaf6 0.0s
=> => naming to docker.io/library/my-nginx:latest              0.0s
=> => unpacking to docker.io/library/my-nginx:latest           0.0s
$ docker run -d my-nginx
e14619a9cbe427e87b80778d0b2bf9737c66fe55d6363810298e322b8995623a
$ docker exec -it e146 bash
root@e14619a9cbe4:/# cat /usr/share/nginx/html/index.html
Hello, OS46!
root@e14619a9cbe4:/# exit
exit
$
```

Problem 3:

- Create a reactjs simple app
- Create a dockerfile to containerize the reactapp
- Build the image and test it
- create a dockerfile for the same app in smaller size using multi staging

```
angular-app > dockerfile.simple > ...
1 FROM node
2
3 WORKDIR /app
4
5 COPY . .
6
7 RUN npm install
8
9 CMD [ "npm", "run", "serve", "--host", "0.0.0.0" ]
10
```

```
.dockerignore X
angular-app > .dockerignore
1 node_modules/
2 docker*
3
```



docker build -t angular-heavy -f dockerfile.heavy .

```
$ docker run -d -p 4200:4200 angular-heavy
64730bd6155cc3a57d07f6ce6cab9882310d387d3f4d656b368fca822fcd74f7
$ curl localhost:4200
<!doctype html>
<html lang="en">
<head>
  <script type="module" src="/@vite/client"></script>

  <meta charset="utf-8">
  <title>AngularApp</title>
  <base href="/">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" type="image/x-icon" href="favicon.ico">
<link rel="stylesheet" href="styles.css"></head>
<body>
  <app-root></app-root>
```

```
dockerfile.simple X
angular-app > dockerfile.simple > ...
1 FROM node:20-alpine AS build
2
3 WORKDIR /app
4
5 COPY . .
6
7 RUN npm install
8 RUN npm run build --configuration=production
9
10 FROM nginx:stable-alpine
11
12 COPY --from=build /app/dist/angular-app /usr/share/nginx/html
13
14 EXPOSE 80
15
16 CMD ["nginx", "-g", "daemon off;"]
17
```

docker build -t angular-simple -f dockerfile.simple .

```
ahmed@HP:Day2/angular-app (main)$ docker run -d -p 8080:80 angular-simple
e65ed9f97d901e31a6086682317c16b3b39b1e7b786e4eab0a6d4ee1ff6b2416
ahmed@HP:Day2/angular-app (main)$ curl localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>
```